

SOFTWARE ENGINEER

The Interview Project

"Build a Real-Time Multi-Tenant Workflow Orchestration Engine"

Codename: FlowForge

The Scenario

You are the founding engineer at a startup building **FlowForge** — a platform where teams can define, execute, monitor, and collaborate on automated workflows in real time. Think of it as a simplified, self-hosted blend of Zapier's workflow engine and GitHub Actions' execution model.

You have **4 days** to deliver a working MVP. You will be evaluated not just on whether it works, but on **how** you build it — your architectural decisions, trade-offs, code quality, and operational thinking.

Core Requirements

A. Workflow Definition & Execution Engine *(Tests: Core Programming, Problem-Solving)*

Build a backend engine that can:

- Accept workflow definitions as a directed acyclic graph (DAG) of steps, where each step is a task with a type (HTTP call, script execution, delay/wait, conditional branch)
 - Parse, validate, and topologically sort the DAG to determine execution order
 - Execute workflows respecting dependencies — parallel where possible, sequential where required
 - Handle step failures with configurable retry logic (exponential backoff, max retries) and a global workflow timeout
-

B. Multi-Tenant API Layer *(Tests: Networking & Protocols, Security)*

Expose a RESTful API (bonus: also offer a GraphQL endpoint) that supports:

- Full CRUD for workflows, with versioning (users can roll back to previous definitions)
 - Workflow triggering (manual, scheduled via cron expression, or webhook-triggered)
 - Pagination, filtering, and rate limiting on all list endpoints
 - Multi-tenant isolation — each tenant's data is strictly separated
 - Authentication via JWT with role-based access control (Admin, Editor, Viewer)
 - All inputs must be validated and sanitized against injection and malformed payloads
-

C. Real-Time Monitoring Dashboard *(Tests: System Design, Performance & Reliability)*

Build a frontend dashboard that:

- Shows live workflow execution status using WebSocket or SSE — steps light up as they start, succeed, or fail in real time

- Displays a visual DAG rendering of any workflow definition
- Provides a run history view with logs, duration, and outcome per step
- Includes a global health panel showing active runs, success/failure rates, and average execution time over the last 24 hours
- Implements client-side caching and optimistic UI updates where appropriate

D. Data Layer *(Tests: Databases & Data Management)*

Design and implement:

- A relational schema for tenants, users, workflow definitions (with version history), and run records
- A separate store (or strategy) for high-volume execution logs (justify your choice: relational, document store, append-only log, etc.)
- At least one non-trivial query optimization — demonstrate with an EXPLAIN plan and your reasoning
- A data migration strategy — include at least one migration script that alters the schema safely

E. Infrastructure & Deployment *(Tests: DevOps, Infrastructure & Cloud)*

Provide:

- A Dockerfile (multi-stage build) for the backend, and a docker-compose.yml that spins up the entire stack locally (API, database, frontend, message broker if used)
- A CI pipeline configuration (GitHub Actions, GitLab CI, or equivalent) that runs linting, tests, builds, and produces a deployable artifact
- A brief infrastructure design document: if you had to deploy this to production on AWS/GCP, draw the architecture and explain your choices (load balancing, auto-scaling, managed services, etc.)

F. Code Quality & Engineering Practices *(Tests: Software Engineering Practices)*

Your submission must demonstrate:

- A clean Git history with meaningful, atomic commits and at least one feature branch with a self-authored pull request (with description)
- Unit tests for the DAG parser and execution engine, integration tests for the API, and at least one end-to-end test for a full workflow run
- A code review exercise: a file called REVIEW.md where you review a deliberately flawed code snippet (provided separately) and write your feedback as if reviewing a teammate's PR
- A README.md with setup instructions, architecture overview, and a section on trade-offs you made and what you would improve with more time

G. AI-Powered Enhancement *(Tests: Emerging & Specialized Knowledge)*

Add one AI-powered feature (choose one or propose your own):

- **Natural language workflow builder** — a user describes a workflow in plain English and the system generates a valid DAG definition using an LLM API
- **Intelligent failure analysis** — when a workflow fails, send the error context to an LLM and surface a suggested fix or diagnosis to the user

- **Smart scheduling** — use historical run data to suggest optimal scheduling windows that avoid peak load

Document your prompt engineering approach, how you handle token limits, and how you guard against malformed LLM output.